Exercise Sheet 4

Programming Languages and Compilers
Prof. Dr. Roland Leißa
Al-Harith Farhad, M.Sc

Exercise 4.1 From a Grammar to an LL(1) Parser

Consider the following grammar:

$$
\begin{aligned}
S &\to \textbf{id}(L) \mid S; \textbf{id}(L) \\
L &\to \varepsilon \mid M \\
M &\to \textbf{id} \mid M, \textbf{id}
\end{aligned}
$$

1. Eliminate left recursion in this grammar.
2. Compute the $First_1$ and $Follow_1$ sets of each non-terminal of your grammar.
3. Construct the parser table for an LL(1) parser for your grammar. Remember to *extend* your grammar.
4. Using your table, give a successful run of the stack deterministic PDA on the input "**id**();**id**(**id**, **id**)#".

Exercise 4.2 Item Pushdown Automaton

Consider the grammar $G = (\{S, A, B\}, \{a, b, c\}, \to, S)$ with $\to$ defined as follows:

$$
\begin{aligned}
S &\to aB \\
A &\to B \mid \varepsilon \\
B &\to bBa \mid aA
\end{aligned}
$$

Give a successful run of the item pushdown automaton of G on the input "abaa". You do *not* have to write down the definition of the item pushdown automaton.

Assume a synthetic start symbol S' which produces S. In each step, do either apply an expansion, a shift or a reduction. The run should end with the stack $[S' \to S.]$.

It is ok to just leave gaps on the stack where the corresponding entry did not change in the corresponding step.